실습 클래스 정답 코드 모음집

설명 주석판

주차별로 독립된 코드이며, 각 public 클래스는 같은 이름의 .java 파일에 둡니다.

문제 PDF · 주차별 소스 · 자료.zip의 ch07/08을 대조하여 정리했습니다.

03주차 이진 트리 기본 연산

높이: 빈 트리 -1, 리프 0. 수식 계산은 정수 연산.

BTNode.java

```
public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;
    // 기본 생성자: 참조형 필드는 기본값 null로 초기화된다.
    public BTNode() {
    }
    // 데이터만 저장하는 리프 노드를 만든다.
    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }
    // 왼쪽 자식, 데이터, 오른쪽 자식을 연결한다.
    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
```

BinaryTree.java

```
public class BinaryTree {
    BTNode root;
    // root가 null인 빈 트리를 만든다.
    public BinaryTree() {
        root = null;
    }
    // 새 루트에 두 서브트리를 연결한다. 기존 노드를 공유한다.
    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }
    // 루트가 없으면 빈 트리이다.
    public boolean isEmpty() {
        return root == null;
    }
```

3주차 | BinaryTree.java · 계속

```
    // 왼쪽 자식을 루트로 하는 트리 객체를 반환한다.
    // 노드를 복제하는 깊은 복사가 아니라 기존 노드를 공유한다.
    public BinaryTree leftSubTree() {
        BinaryTree tmp = new BinaryTree();
        if (root != null) {
            tmp.root = root.Lchild;
        }
        return tmp;
    }
    // 오른쪽 자식을 루트로 하는 트리 객체를 반환한다.
    public BinaryTree rightSubTree() {
        BinaryTree tmp = new BinaryTree();
        if (root != null) {
            tmp.root = root.Rchild;
        }
        return tmp;
    }
    // 루트 노드에 저장된 문자열을 반환한다.
    public String rootData() {
        if (root == null) {
            System.out.println("빈루트");
            return null;
        }
        return root.data;
    }
    // 루트에서 재귀 계산을 시작한다. 빈 수식은 예외로 처리한다.
    public int calculate() {
        if (root == null) {
            throw new IllegalStateException("Empty expression tree");
        }
        return theCalculate(root);
    }
    // 리프는 정수, 내부 노드는 두 피연산자를 갖는 연산자이다.
    // 왼쪽과 오른쪽을 먼저 계산한 뒤 연산하는 후위 처리이다.
    private int theCalculate(BTNode t) {
        if (t.Lchild == null && t.Rchild == null) {
            // 숫자 문자열을 int로 바꾼다.
            return Integer.parseInt(t.data);
        }
        // 왼쪽 피연산자부터 재귀 계산한다.
        int left = theCalculate(t.Lchild);
        // 오른쪽 피연산자를 재귀 계산한다.
        int right = theCalculate(t.Rchild);
        switch (t.data) {
            case "+": return left + right;
            case "-": return left - right;
            case "*": return left * right;
            // int 나눗셈이므로 소수 부분은 버린다.
            case "/": return left / right;
            default:
                throw new IllegalArgumentException("Unknown operator");
        }
    }
    // 루트에서 가장 먼 리프까지의 간선 수를 구한다.
    public int height() {
        return theHeight(root);
    }
```

3주차 | BinaryTree.java · 계속

```
    // 빈 트리 -1, 리프 0: 강의의 루트 레벨 0 기준이다.
    private int theHeight(BTNode t) {
        if (t == null) {
            return -1;
        }
        int leftHeight = theHeight(t.Lchild);
        int rightHeight = theHeight(t.Rchild);
        // 더 깊은 쪽 높이에 현재 간선 1을 더한다.
        return 1 + Math.max(leftHeight, rightHeight);
    }
    // 수식을 출력한 뒤 줄을 바꾼다.
    public void print0() {
        print0(root);
        System.out.println();
    }
    // 리프는 숫자만, 내부 노드는 (왼쪽 수식 연산자 오른쪽 수식).
    private void print0(BTNode t) {
        if (t == null) return;
        if (t.Lchild == null && t.Rchild == null) {
            System.out.print(t.data);
            return;
        }
        System.out.print("(");
        print0(t.Lchild);
        System.out.print(t.data);
        print0(t.Rchild);
        System.out.print(")");
    }
}
```

05주차 순회 · 복사 · 동등 비교

Stack·Queue는 제공된 보조 클래스이며 뒤에 함께 수록.

BTNode.java

```
public class BTNode {
    String data;
    BTNode Lchild;
    BTNode Rchild;
    // 기본 생성자: 참조형 필드는 기본값 null로 초기화된다.
    public BTNode() {
    }
    // 데이터만 저장하는 리프 노드를 만든다.
    public BTNode(String dt) {
        data = dt;
        Lchild = null;
        Rchild = null;
    }
    // 왼쪽 자식, 데이터, 오른쪽 자식을 연결한다.
    public BTNode(BTNode lc, String dt, BTNode rc) {
        data = dt;
        Lchild = lc;
        Rchild = rc;
    }
}
```

BinaryTree.java

```java
public class BinaryTree {
    BTNode root;
    // root가 null인 빈 트리를 만든다.
    public BinaryTree() {
        root = null;
    }
    // 새 루트에 두 서브트리를 연결한다. 기존 노드를 공유한다.
    public BinaryTree(BinaryTree ltree, String data, BinaryTree rtree) {
        root = new BTNode(ltree.root, data, rtree.root);
    }
    // 루트가 없으면 빈 트리이다.
    public boolean isEmpty() {
        return root == null;
    }
    // 왼쪽 자식을 루트로 하는 트리 객체를 반환한다.
    // 노드를 복제하는 깊은 복사가 아니라 기존 노드를 공유한다.
    // 제공 코드의 전제: 빈 트리에서는 호출하지 않는다.
    public BinaryTree leftSubTree() {
        BinaryTree ltree = new BinaryTree();
        ltree.root = root.Lchild;
        return ltree;
    }
    // 오른쪽 자식을 루트로 하는 트리 객체를 반환한다.
    // 제공 코드의 전제: 빈 트리에서는 호출하지 않는다.
    public BinaryTree rightSubTree() {
        BinaryTree rtree = new BinaryTree();
        rtree.root = root.Rchild;
        return rtree;
    }
    // 루트 노드에 저장된 문자열을 반환한다.
    // 제공 코드의 전제: 빈 트리에서는 호출하지 않는다.
    public String rootData() {
        return root.data;
    }
    // 문제에서 제공한 호출 함수: 재귀와 반복 중위를 모두 출력한다.
    public void inorder() {
        System.out.println();
        System.out.println("Inorder");
        theInorder(root);
        System.out.println();
        System.out.println("InorderIter");
        inorderIter1();
    }
    // 문제에서 제공한 호출 함수: 재귀와 반복 전위를 모두 출력한다.
    public void preorder() {
        System.out.println("preorder");
        System.out.println();
        thePreorder(root);
        System.out.println();
        System.out.println("preorderIter");
        preorderIter();
    }
    // 루트에서 재귀 후위 순회를 시작한다.
    public void postorder() {
        thePostorder(root);
    }
```

5주차 | BinaryTree.java · 계속

```
    // 중위 L-V-R: 왼쪽 -> 현재 노드 -> 오른쪽.
    // t가 null이면 아무 작업 없이 재귀가 끝난다.
    private void theInorder(BTNode t) {
        if (t != null) {
            theInorder(t.Lchild);
            System.out.print(t.data);
            theInorder(t.Rchild);
        }
    }
    // 전위 V-L-R: 현재 노드 -> 왼쪽 -> 오른쪽.
    private void thePreorder(BTNode t) {
        if (t != null) {
            System.out.print(t.data);
            thePreorder(t.Lchild);
            thePreorder(t.Rchild);
        }
    }
    // 후위 L-R-V: 왼쪽 -> 오른쪽 -> 현재 노드.
    private void thePostorder(BTNode t) {
        if (t != null) {
            thePostorder(t.Lchild);
            thePostorder(t.Rchild);
            System.out.print(t.data);
        }
    }
    // 스택은 나중에 방문할 조상 노드를 기억한다.
    // 왼쪽 끝까지 내려간 뒤 꺼내어 방문하고 오른쪽으로 이동한다.
    private void inorderIter1() {
        Stack stack = new Stack();
        BTNode current = root;
        // 현재 노드 또는 돌아갈 조상이 남아 있으면 계속한다.
        while (current != null || !stack.empty()) {
            while (current != null) {
                stack.push(current);
                current = current.Lchild;
            }
            // Object 반환값을 BTNode로 형 변환한다.
            current = (BTNode) stack.pop();
            System.out.print(current.data);
            current = current.Rchild;
        }
    }
```

5주차 | BinaryTree.java · 계속

```
    // 스택의 LIFO 특성을 이용한 전위 순회이다.
    private void preorderIter() {
        if (root == null)
            return;
        Stack stack = new Stack();
        stack.push(root);
        while (!stack.empty()) {
            BTNode current = (BTNode) stack.pop();
            System.out.print(current.data);
            // 오른쪽을 먼저 넣어 왼쪽이 먼저 나오게 한다.
            if (current.Rchild != null)
                stack.push(current.Rchild);
            // 왼쪽을 나중에 push하면 먼저 방문한다.
            if (current.Lchild != null)
                stack.push(current.Lchild);
        }
    }
    // 큐의 FIFO 특성으로 같은 레벨을 왼쪽부터 방문한다.
    public void levelorder() {
        if (root == null)
            return;
        Queue queue = new Queue();
        queue.enqueue(root);
        while (!queue.isEmpty()) {
            BTNode current = (BTNode) queue.dequeue();
            System.out.print(current.data);
            if (current.Lchild != null)
                queue.enqueue(current.Lchild);
            if (current.Rchild != null)
                queue.enqueue(current.Rchild);
        }
    }
    // 새 트리 객체를 만들고 노드들도 재귀적으로 새로 생성한다.
    public BinaryTree copy() {
        BinaryTree newTree = new BinaryTree();
        newTree.root = theCopy(root);
        return newTree;
    }
    // null은 그대로 반환하고, 실제 노드는 각각 새로 만든다.
    // 문자열은 불변이므로 data는 공유해도 된다.
    private BTNode theCopy(BTNode t) {
        if (t == null)
            return null;
        BTNode newNode = new BTNode(
            theCopy(t.Lchild), t.data, theCopy(t.Rchild));
        return newNode;
    }
    // 전제: tr은 null이 아닌 BinaryTree 객체이다.
    // 두 루트부터 구조와 데이터 값을 비교한다.
    public boolean equals(BinaryTree tr) {
        return theEqual(this.root, tr.root);
    }
```

5주차 | BinaryTree.java · 계속

```
    // 둘 다 null이면 같은 빈 구조, 하나만 null이면 다른 구조이다.
    private boolean theEqual(BTNode s, BTNode t) {
        if (s == null && t == null)
            return true;
        if (s == null || t == null)
            return false;
        // 문자열 내용은 ==가 아닌 equals로 비교한다.
        if (!s.data.equals(t.data))
            return false;
        return theEqual(s.Lchild, t.Lchild)
            // 왼쪽과 오른쪽이 모두 같아야 같은 트리이다.
            && theEqual(s.Rchild, t.Rchild);
    }
}
```

Stack.java

```
public class Stack {
    private int top;
    private Object[] data;
    private final static int MAX = 100;
    // 제공된 배열 스택: top은 마지막 원소 위치, -1은 빈 상태.
    public Stack() {
        data = new Object[MAX];
        top = -1;
    }
    // 현재 저장된 원소 수를 반환한다.
    public int size() {
        return top + 1;
    }
    // top == -1이면 저장된 원소가 없다.
    public boolean empty() {
        return top == -1;
    }
    // top을 먼저 증가시키고 새 원소를 저장한다.
    public void push(Object x) {
        top = top + 1;
        data[top] = x;
    }
    // 마지막 원소를 반환한다. 호출 전 empty()로 확인해야 한다.
    public Object pop() {
        top = top - 1;
        return data[top + 1];
    }
    // 삭제하지 않고 스택의 맨 위 원소를 확인한다.
    public Object peek() {
        return data[top];
    }
}
```

Queue.java

```
public class Queue {
    private int front;
    private int back;
    private Object[] data;
    private final static int MAX = 100;
```

5주차 | Queue.java · 계속

```
    // 제공된 원형 큐: front는 첫 원소 바로 앞, back은 마지막 위치.
    public Queue() {
        data = new Object[MAX];
        front = back = 0;
    }
    // 현재 저장된 원소 수를 반환한다.
    public int size() {
        if (back >= front)
            return back - front;
        else
            return MAX + back - front;
    }
    // front == back이면 빈 큐이다. 한 칸을 비워 최대 99개 저장.
    public boolean isEmpty() {
        return front == back;
    }
    // back을 한 칸 순환 이동한 뒤 저장한다.
    // 제공 코드에는 가득 찼을 때의 예외 처리가 없다.
    public void enqueue(Object x) {
        back = (back + 1) % MAX;
        if (back == front) {
        }
        data[back] = x;
    }
    // front를 한 칸 이동하여 가장 먼저 들어온 원소를 반환한다.
    // 빈 큐에서는 호출하지 않는다.
    public Object dequeue() {
        front = (front + 1) % MAX;
        return data[front];
    }
    // 삭제 없이 첫 원소를 확인한다.
    public Object front() {
        return data[(front + 1) % MAX];
    }
}
```

07주차 이진 탐색 트리

BSTsplit은 강의의 연결 변경 방식. 분할 후 원본은 빈 트리.

TreeNode.java

```
public class TreeNode {
    String key;
    TreeNode Lchild;
    TreeNode Rchild;
}
```

BinarySearchTree.java

```
public class BinarySearchTree {
    private TreeNode rootNode;
```

7주차 | BinarySearchTree.java · 계속

```
    // 현재 키와 비교: 작으면 왼쪽, 크면 오른쪽으로 탐색한다.
    public TreeNode BSTsearch(String K) {
        TreeNode p = rootNode;
        while (p != null) {
            // 음수: K가 작음 / 0: 같음 / 양수: K가 큼.
            int result = K.compareTo(p.key);
            if (result == 0)
                return p;
            else if (result < 0)
                p = p.Lchild;
            else
                p = p.Rchild;
        }
        return null;
    }
    // 검색 경로를 따라 빈 위치를 찾고 부모에 새 노드를 연결한다.
    // 같은 키는 중복 삽입하지 않는다.
    public void BSTinsert(String K) {
        TreeNode newNode = new TreeNode();
        newNode.key = K;
        newNode.Lchild = null;
        newNode.Rchild = null;
        if (rootNode == null) {
            rootNode = newNode;
            return;
        }
        TreeNode p = rootNode;
        // p가 null에 도달해도 직전 부모를 기억한다.
        TreeNode parent = null;
        while (p != null) {
            parent = p;
            // 음수: K가 작음 / 0: 같음 / 양수: K가 큼.
            int result = K.compareTo(p.key);
            if (result == 0)
                return;
            else if (result < 0)
                p = p.Lchild;
            else
                p = p.Rchild;
        }
        if (K.compareTo(parent.key) < 0)
            parent.Lchild = newNode;
        else
            parent.Rchild = newNode;
    }
    // BST의 가장 큰 키는 오른쪽 끝 노드에 있다.
    private TreeNode maxNode(TreeNode root) {
        if (root == null)
            return null;
        while (root.Rchild != null) {
            root = root.Rchild;
        }
        return root;
    }
```

7주차 | BinarySearchTree.java · 계속

```
    // 삭제 후 이 서브트리의 새 루트를 반환한다.
    // 자식 0개/1개는 자식으로 대체, 2개는 왼쪽 최댓값으로 대체.
    private TreeNode delete(TreeNode root, String K) {
        if (root == null)
            return null;
        int result = K.compareTo(root.key);
        if (result < 0) {
            root.Lchild = delete(root.Lchild, K);
        } else if (result > 0) {
            root.Rchild = delete(root.Rchild, K);
        } else {
            // 리프이면 null, 오른쪽만 있으면 오른쪽 자식을 반환한다.
            if (root.Lchild == null)
                return root.Rchild;
            // 왼쪽 자식만 남은 경우이다.
            if (root.Rchild == null)
                return root.Lchild;
            // 중위 선행자: 왼쪽 서브트리의 최댓값.
            TreeNode temp = maxNode(root.Lchild);
            // 노드 전체가 아니라 키를 복사한다.
            root.key = temp.key;
            // 복사한 원래 선행자 노드를 제거하고 다시 연결한다.
            root.Lchild = delete(root.Lchild, temp.key);
        }
        return root;
    }
    // 루트 자체가 삭제될 수 있으므로 반환값을 rootNode에 저장한다.
    public void BSTdelete(String K) {
        rootNode = delete(rootNode, K);
    }
```

7주차 | BinarySearchTree.java · 계속

```
    // 강의의 연결 변경 방식: 탐색 경로만 따라가므로 O(h).
    // SmallTree와 LargeTree는 원본과 서로 다른 객체여야 한다.
    // 기준 x는 두 결과에서 제외한다. x가 존재했는지를 반환한다.
    // 원본 노드를 결과에 옮긴 뒤 원본은 빈 트리로 만든다.
    public boolean BSTsplit(String x,
                            BinarySearchTree SmallTree,
                            BinarySearchTree LargeTree) {
        // 더미 노드: Small.Rchild가 작은 결과 트리의 루트.
        TreeNode Small = new TreeNode();
        // 더미 노드: Large.Lchild가 큰 결과 트리의 루트.
        TreeNode Large = new TreeNode();
        TreeNode S = Small;
        TreeNode L = Large;
        TreeNode P = rootNode;
        boolean found = false;
        while (P != null) {
            int result = x.compareTo(P.key);
            if (result == 0) {
                // x를 찾음: x의 왼쪽 서브트리는 작은 쪽에 붙인다.
                S.Rchild = P.Lchild;
                // x의 오른쪽 서브트리는 큰 쪽에 붙인다.
                L.Lchild = P.Rchild;
                found = true;
                break;
            } else if (result < 0) {
                // x < P.key: P와 그 오른쪽 부분은 큰 쪽이다.
                L.Lchild = P;
                L = P;
                P = P.Lchild;
            } else {
                // x > P.key: P와 그 왼쪽 부분은 작은 쪽이다.
                S.Rchild = P;
                S = P;
                P = P.Rchild;
            }
        }
        // x가 없으면 양쪽 끝의 기존 연결을 끊는다.
        if (!found) {
            S.Rchild = null;
            L.Lchild = null;
        }
        SmallTree.rootNode = Small.Rchild;
        LargeTree.rootNode = Large.Lchild;
        // 원본은 비우고 두 결과 트리에 소유권을 넘긴다.
        rootNode = null;
        return found;
    }
    // 괄호를 포함한 중위 순회로 BST의 구조를 출력한다.
    private void printNode(TreeNode n) {
        if (n != null) {
            System.out.print("(");
            printNode(n.Lchild);
            System.out.print(" " + n.key + " ");
            printNode(n.Rchild);
            System.out.print(")");
        }
    }
```

7주차 | BinarySearchTree.java · 계속

```
    // 루트부터 출력하고 줄을 바꾼다.
    public void print() {
        printNode(rootNode);
        System.out.println();
    }
}
```

08주차 최대 힙

실습 출력 기준으로 입력 배열 origArray도 1번부터 사용(0번 제외).
문자열 compareTo()로 비교. 배열 생성자는 아래에서 위로 힙화.

Heap.java

```
public class Heap {
    private int count;
    private int size;
    private String[] itemArray;
    // 배열 0번은 사용하지 않는다. count는 현재 원소 수이다.
    public Heap() {
        count = 0;
        size = 64;
        itemArray = new String[size];
    }
```

8주차 | Heap.java · 계속

```
    // 문제 실행결과 기준: origArray[0]은 사용하지 않는다.
    // 1번부터 복사한 뒤 마지막 부모부터 아래로 내려 O(n)에 힙화한다.
    public Heap(String[] origArray) {
        count = Math.max(0, origArray.length - 1);
        size = Math.max(64, count + 1);
        itemArray = new String[size];
        for (int i = 1; i <= count; i++) {
            itemArray[i] = origArray[i];
        }
        // 리프는 이미 힙이다. 마지막 내부 노드부터 처리한다.
        for (int i = count / 2; i >= 1; i--) {
            int parent = i;
            int child = parent * 2;
            String temp = itemArray[parent];
            while (child <= count) {
                // 오른쪽 자식이 존재할 때만 두 자식을 비교한다.
                if (child < count &&
                    itemArray[child].compareTo(itemArray[child + 1]) < 0) {
                    // 더 큰 오른쪽 자식을 선택한다.
                    child++;
                }
                // 내려갈 값이 큰 자식 이상이면 제자리이다.
                if (temp.compareTo(itemArray[child]) >= 0) {
                    break;
                }
                // 큰 자식을 위 빈자리로 올린다.
                itemArray[parent] = itemArray[child];
                parent = child;
                // 한 레벨 내려가 다시 왼쪽 자식부터 비교한다.
                child = parent * 2;
            }
            itemArray[parent] = temp;
        }
    }
    // 끝에 새 자리를 확보하고 부모보다 크면 위로 올린다.
    public void insert(String newItem) {
        // 1번부터 사용하므로 배열 길이보다 원소 수가 1 작아야 한다.
        if (count + 1 >= size) {
            size *= 2;
            String[] temp = new String[size];
            System.arraycopy(itemArray, 0, temp, 0, count + 1);
            itemArray = temp;
        }
        int i = ++count;
        // 부모 인덱스는 i/2. 루트에 닿거나 부모가 크면 종료한다.
        while (i > 1 && itemArray[i / 2].compareTo(newItem) < 0) {
            // 작은 부모를 아래로 내려 빈자리를 위로 이동한다.
            itemArray[i] = itemArray[i / 2];
            i /= 2;
        }
        // 최종 위치에 새 값을 한 번 저장한다.
        itemArray[i] = newItem;
    }
```

8주차 | Heap.java · 계속

```
    // 루트의 최댓값을 반환하고 마지막 원소를 아래로 내려 재배치한다.
    public String delete() {
        if (count == 0) {
            return null;
        }
        // 삭제하여 반환할 최댓값을 보관한다.
        String max = itemArray[1];
        // 마지막 원소를 루트 자리의 후보로 잡는다.
        String last = itemArray[count];
        itemArray[count] = null;
        count--;
        int parent = 1;
        int child = 2;
        while (child <= count) {
            // 오른쪽 자식이 존재할 때만 두 자식을 비교한다.
            if (child < count &&
                itemArray[child].compareTo(itemArray[child + 1]) < 0) {
                // 더 큰 오른쪽 자식을 선택한다.
                child++;
            }
            // last가 큰 자식 이상이면 더 내려갈 필요가 없다.
            if (last.compareTo(itemArray[child]) >= 0) {
                break;
            }
            // 큰 자식을 위 빈자리로 올린다.
            itemArray[parent] = itemArray[child];
            parent = child;
            // 한 레벨 내려가 다시 왼쪽 자식부터 비교한다.
            child = parent * 2;
        }
        // 마지막 하나를 지운 경우에는 다시 저장하지 않는다.
        if (count > 0) {
            itemArray[parent] = last;
        }
        return max;
    }
}
```